Git remoto, sincronización y flujo colaborativo

Introducción

En la sesión anterior trabajaste Git en local: comprendiste la diferencia entre workspace, index y repositorio local, aprendiste a construir commits con intención y diste los primeros pasos con ramas y merges sencillos. Esa base sigue siendo imprescindible, pero todavía falta una pieza clave del trabajo real con Git: la dimensión remota y colaborativa.

En esta sesión vas a aprender cómo se relaciona tu repositorio local con un repositorio remoto, qué significa sincronizar cambios, cómo se publica trabajo en GitHub y por qué no conviene trabajar directamente sobre la rama principal cuando el proyecto empieza a ser compartido.

También vas a introducir el flujo de trabajo que se utilizará en el bootcamp: partir de una rama principal actualizada, abrir una rama de tarea, trabajar en local, publicar esa rama, revisar el trabajo y preparar su integración. El objetivo no es que memorices tres o cuatro comandos nuevos, sino que entiendas cómo se amplía el modelo mental de Git cuando pasas del entorno individual al entorno compartido.

---

1. Del repositorio local al repositorio remoto

Hasta ahora has trabajado con repositorios que vivían únicamente en tu equipo. Ese modelo es útil para aprender la base, pero en proyectos reales el código necesita existir también en un espacio compartido.

1.1. Qué es un repositorio remoto

Un **repositorio remoto** es otra copia del repositorio, alojada fuera de tu máquina y accesible normalmente por red.

Ese remoto te permite:

- compartir el proyecto con otras personas;
- publicar tus commits;
- recuperar cambios externos;
- construir una referencia común del estado del proyecto.

1.2. Qué no debes imaginar

No debes pensar que el remoto “sustituye” a tu repositorio local. Tampoco debes imaginar que local y remoto son exactamente el mismo sitio.

Lo correcto es entender que:

- tu repositorio local y el remoto están relacionados;
- ambos pueden tener estados distintos durante un tiempo;
- la sincronización entre ambos no ocurre automáticamente;
- Git te obliga a decidir cuándo traes o publicas cambios.

> **Idea clave**
>
> Git remoto no elimina Git local. Lo amplía hacia un contexto compartido.

1.3. Qué cambia cuando el proyecto pasa a ser compartido

Cuando aparece un remoto, tu trabajo deja de ser una secuencia cerrada en tu máquina y pasa a formar parte de una dinámica más amplia. Ahora debes pensar también en:

- qué has publicado;
- qué trabajo ajeno existe ya en el remoto;
- si tu rama está actualizada;
- cuándo te conviene integrar cambios;
- cómo evitar sobrescribir o mezclar trabajo de forma desordenada.

Referencias del subtema

- Git Book: remotos.
- Documentación oficial de Git.
- GitHub Docs.

---

2. Git y GitHub no son lo mismo

Es importante distinguir claramente entre la herramienta y la plataforma.

Concepto	Qué es	Para qué sirve
Git	Sistema de control de versiones	Registrar historial, crear ramas, integrar cambios, sincronizar repositorios
GitHub	Plataforma que aloja repositorios Git	Compartir, revisar, colaborar, gestionar repositorios remotos

2.1. Por qué esta distinción importa

Si confundes Git con GitHub, puedes acabar pensando que trabajar con Git “es subir cosas a GitHub”. Eso es incorrecto.

Git sigue funcionando plenamente en local, aunque no uses GitHub. Lo que GitHub aporta es una capa de colaboración, visibilidad y gestión sobre repositorios Git.

> **Regla mental útil**
>
> Git es la herramienta. GitHub es uno de los entornos donde esa herramienta puede compartirse y gestionarse.

Referencias del subtema

- Git Book: introducción a GitHub.
- GitHub Docs.
- Documentación oficial de Git.

---

3. Relación entre local y remoto

Para entender bien la sesión, necesitas fijar la relación entre varios niveles del flujo.

3.1. Mapa general del trabajo con Git

Nivel	Qué representa	Quién lo controla directamente
Workspace	Tus archivos modificados en el directorio de trabajo	Tú
Index / Staging	Cambios preparados para el siguiente commit	Tú

Nivel	Qué representa	Quién lo controla directamente
Repositorio local	Historial confirmado en tu máquina	Tú
Repositorio remoto	Historial compartido en el servidor	Tú y otras personas del equipo

3.2. Qué consecuencias tiene este mapa

De este esquema se deduce algo fundamental: tu historial local puede estar por delante, por detrás o divergido respecto al remoto.

Eso significa que:

- puedes tener commits locales que todavía no has publicado;
- puede haber commits remotos que todavía no has traído;
- puedes estar trabajando en una rama local que aún no existe en remoto;
- puede existir una rama remota que tú todavía no has convertido en rama local propia de trabajo.

> ⚠ **Precisión importante**
>
> No des por hecho que lo que ves en tu equipo coincide siempre con lo que existe en el remoto.

Referencias del subtema

- Git Book: ramas remotas y remotos.
- Documentación oficial de Git.
- Explain Git with D3.

---

4. Obtener un proyecto existente con `git clone`

Cuando un proyecto ya existe en remoto, lo habitual no es crear el repositorio desde cero con `git init`, sino **clonarlo**.

```
git clone URL_DEL_REPOSITORIO
```

4.1. Qué hace `git clone`

`git clone` crea una copia local operativa del repositorio remoto. Esa copia incluye:

- archivos del proyecto;
- historial;
- referencias a ramas;
- configuración inicial del remoto.

4.2. Por qué no equivale a descargar un `.zip`

Descargar una carpeta comprimida te da archivos, pero no te da un repositorio Git funcional. Clonar, en cambio, te permite seguir trabajando con historial, ramas y sincronización.

4.3. Qué ocurre después de clonar

Tras clonar un proyecto, normalmente ya tienes configurado un remoto principal con un nombre por defecto: `origin`.

Ese nombre no es mágico, pero sí es la convención más habitual.

> **Idea importante**
>
> `git clone` no trae solo la última versión de los archivos. Trae una copia funcional del repositorio.

Referencias del subtema

- Documentación oficial: `git clone`.
- Git Book: obtención de repositorios.
- GitHub Docs.

---

5. Consultar remotos con `git remote`

Git te permite inspeccionar los remotos configurados en el repositorio.

```
git remote -v
```

5.1. Qué te muestra este comando

Te muestra los nombres de los remotos y sus direcciones asociadas.

5.2. Por qué te interesa mirarlo

Este comando te ayuda a saber:

- si el repositorio tiene remoto configurado;
- qué remoto es el principal;
- a qué dirección apunta;
- si estás trabajando con el repositorio correcto.

5.3. `origin` como convención

Cuando clonas un repositorio, Git suele asignar el nombre `origin` al remoto principal.

Nombre habitual	Qué suele representar
`origin`	Repositorio remoto principal del proyecto
otros nombres	Remotos adicionales o alternativos

> **Criterio práctico**
>
> Aunque `origin` sea habitual, no debes aprenderlo como una palabra mágica. Debes entender que es simplemente un nombre de referencia.

Referencias del subtema

- Documentación oficial: `git remote`.
- Git Book: inspección de remotos.
- Git Cheat Sheet oficial.

---

6. Publicar cambios con `git push`

Cuando haces commits en local, esos commits siguen viviendo únicamente en tu repositorio local hasta que los publicas.

```
git push
```

6.1. Qué hace `git push`

`git push` envía al remoto commits que ya existen en tu historial local.

6.2. Qué no significa

No significa “guardar más fuerte”.
No significa “hacer que Git termine de funcionar”.
No significa “integrar automáticamente tu trabajo en la rama principal del proyecto”.

Lo que significa es: **publicar en el remoto lo que ya has confirmado en local**.

6.3. Por qué el contexto importa

Antes de hacer `push`, te interesa tener claro:

- en qué rama estás;
- qué commits vas a publicar;
- si tu rama representa una tarea coherente;
- si esa publicación debe ir a una rama de trabajo y no a la principal.

> ⚠ **Precaución**
>
> No conviertas `git push` en un gesto mecánico. Primero debes saber qué estás publicando y dónde.

Referencias del subtema

- Documentación oficial: `git push`.
- Git Book: publicación de ramas.
- GitHub Docs.

---

7. Traer información del remoto con `git fetch`

Uno de los comandos más importantes de esta sesión es `git fetch`.

```
git fetch
```

7.1. Qué hace `git fetch`

`git fetch` actualiza tu conocimiento del estado del remoto, trayendo referencias y cambios remotos a tu repositorio local, pero **sin integrarlos automáticamente en tu rama actual**.

7.2. Por qué es valioso

Es valioso porque te permite saber qué ha cambiado en el remoto sin alterar inmediatamente tu trabajo actual.

7.3. Qué te aporta conceptualmente

`fetch` separa dos acciones que muchas personas mezclan:

1. enterarte de que el remoto ha cambiado;
2. incorporar esos cambios a tu línea actual de trabajo.

> ✓ **Idea clave**
>
> `fetch` te informa y actualiza referencias. No modifica automáticamente tu rama de trabajo actual.

Referencias del subtema

- Documentación oficial: `git fetch`.
- Git Book: descarga de referencias remotas.
- Explain Git with D3.

---

8. Traer e integrar con `git pull`

`git pull` se utiliza para traer cambios del remoto e integrarlos en tu rama actual.

```
git pull
```

8.1. Qué hace

De forma simplificada, `git pull` combina dos ideas:

- traer cambios del remoto;
- integrarlos en la rama actual.

8.2. Por qué no deberías usarlo a ciegas

Si usas `pull` automáticamente, puedes alterar tu rama actual sin haber entendido antes qué cambios estaban llegando.

8.3. Cuándo tiene sentido

Tiene sentido cuando:

- sabes en qué rama estás;
- entiendes qué remoto y qué rama quieres actualizar;
- asumes que esa integración va a afectar directamente a tu línea actual de trabajo.

Comando	Qué hace	¿Modifica tu rama actual directamente?
`git fetch`	Trae información del remoto	No
`git pull`	Trae e integra	Sí
`git push`	Publica commits locales	No modifica tu rama local; publica en remoto

> **Criterio de uso**
>
> `git pull` no es “la forma normal de ver si hay cambios”. Antes de usarlo, debes entender qué impacto tendrá en tu rama.

Referencias del subtema

- Documentación oficial: `git pull`.
- Git Book: obtención e integración desde remotos.
- GitHub Docs.

---

9. Diferencia entre `fetch`, `pull` y `push`

Esta tríada es uno de los núcleos conceptuales de la sesión.

9.1. Comparación esencial

Comando	Dirección principal	Función
`git fetch`	remoto -> local	Actualiza referencias remotas sin integrar
`git pull`	remoto -> local	Trae e integra cambios en tu rama actual
`git push`	local -> remoto	Publica tus commits en el remoto

9.2. Regla mental útil

Puedes recordarlo así:

- **fetch**: “quiero saber qué ha cambiado”
- **pull**: “quiero traerlo e incorporarlo”
- **push**: “quiero publicar mi trabajo”

9.3. Error frecuente

Uno de los errores más comunes es usar `pull` cuando en realidad solo querías saber si el remoto había cambiado.

> **Hábito recomendable**
>
> Cuando tengas dudas, piensa primero si necesitas **informarte** o **integrar**. Esa distinción suele aclarar si te conviene más `fetch` o `pull`.

Referencias del subtema

- Git Book: remotos.
- Documentación oficial de Git.
- Git Cheat Sheet oficial.

---

10. Ramas locales y ramas remotas

A partir de esta sesión ya no basta con pensar solo en ramas locales. Necesitas entender también que existen referencias remotas asociadas al trabajo compartido.

10.1. Qué debes entender sin entrar aún en demasiados detalles internos

- una rama local puede existir solo en tu equipo;
- una rama remota puede existir en el servidor aunque tú todavía no trabajes directamente sobre ella como rama local propia;
- una rama local y una rama remota relacionadas no son exactamente “la misma cosa”, aunque representen líneas conectadas del mismo proyecto.

10.2. Por qué esto importa

Cuando trabajas en equipo, ya no piensas solo en “mi rama”, sino también en:

- qué versión compartida existe en el remoto;
- si tu rama local está alineada con esa versión;

- si el remoto contiene trabajo ajeno que tú todavía no has traído.

> **Idea importante**
>
> En Git remoto, las ramas no son solo una herramienta de organización local. También son unidades de publicación y colaboración.

Referencias del subtema

- Git Book: ramas remotas.
- Documentación oficial de Git.
- Explain Git with D3.

---

11. Flujo de trabajo con ramas en GitHub

En este bootcamp no vas a trabajar publicando directamente todo sobre la rama principal.

11.1. Flujo base que se va a utilizar

El flujo recomendado será este:

1. partes de una rama principal actualizada;
2. creas una rama de tarea;
3. trabajas en local sobre esa rama;
4. haces commits con intención;
5. publicas la rama en remoto;
6. preparas revisión;
7. integras cuando corresponda.

11.2. Por qué no conviene trabajar sobre `main`

Trabajar directamente sobre `main` para cada tarea cotidiana genera varios problemas:

- mezcla de cambios;
- dificultad de revisión;
- mayor riesgo de inestabilidad;
- menor claridad del historial.

11.3. Convención básica de nombres

En este bloque te conviene seguir una convención sencilla:

Prefijo de rama	Uso recomendado
`feature/`	Nueva funcionalidad
`fix/`	Corrección
`docs/`	Documentación

11.4. Una rama por tarea

La idea base no es crear ramas por crear, sino usar una rama como contenedor de una unidad lógica de trabajo.

> **Criterio metodológico**
>
> En este bloque, la rama no es un detalle opcional del flujo. Es la base de la organización del trabajo colaborativo.

Referencias del subtema

- GitHub Flow.
- Git Book: organización del trabajo con ramas.
- GitHub Docs.

---

12. Pull Requests como petición de integración

Una Pull Request no es solo un botón de GitHub. Es una propuesta formal de integración.

12.1. Qué representa una Pull Request

Cuando abres una Pull Request, estás diciendo algo parecido a esto:

- he terminado una unidad de trabajo;
- la he desarrollado en una rama concreta;
- propongo integrarla en otra rama;
- quiero que se revise antes de hacerlo.

12.2. Qué valor aporta

La Pull Request permite:

- revisar cambios antes de integrar;
- comentar decisiones técnicas;
- detectar problemas;
- ordenar mejor el flujo del proyecto;
- separar publicación de integración.

12.3. Por qué tiene sentido incluso en contextos formativos

Aunque en esta fase hagáis PRs simuladas o sencillas, su valor pedagógico es alto porque te obliga a:

- delimitar mejor la tarea;
- presentar el trabajo con claridad;
- acostumbrarte a la revisión.

> ✓ **Idea clave**
>
> Una Pull Request no solo une ramas. También comunica intención y ordena el proceso de revisión.

Referencias del subtema

- GitHub Docs: Pull Requests.
- GitHub Flow.
- Recursos de revisión colaborativa.

---

13. Revisión de cambios antes de integrar

Publicar una rama no equivale a darla por válida automáticamente.

13.1. Qué te interesa revisar

Antes de integrar una rama, conviene revisar:

- si la tarea está claramente acotada;
- si los commits son comprensibles;
- si hay cambios accidentales;
- si la rama mezcla varias cosas que deberían haberse separado;

- si el resultado responde al objetivo.

13.2. Qué papel cumple la revisión

La revisión no existe solo para detectar errores. También sirve para:

- mejorar la comunicación técnica;
- justificar decisiones;
- acostumbrarte a leer trabajo ajeno;
- construir criterio profesional.

> **Hábito importante**
>
> Antes de pedir integración, revisa tu propia rama como si fueras otra persona del equipo que va a leerla por primera vez.

Referencias del subtema

- GitHub Docs.
- Buenas prácticas de code review.
- Git Book.

---

14. Conflictos en contexto remoto

Los conflictos no desaparecen cuando pasas al trabajo remoto. De hecho, a menudo se vuelven más frecuentes porque varias personas pueden modificar zonas cercanas del proyecto.

14.1. Cuándo pueden aparecer

Pueden aparecer cuando:

- otra persona ya ha integrado cambios en la rama principal;
- tú has seguido trabajando sobre una rama desactualizada;
- dos ramas modifican de forma incompatible la misma parte del proyecto.

14.2. Qué debes entender

El conflicto remoto no es un fenómeno completamente distinto del local. La lógica sigue siendo la misma: Git detecta una incompatibilidad y te pide una decisión humana.

14.3. Qué cambia respecto al conflicto local

Lo que cambia no es la naturaleza del conflicto, sino el contexto:

- ahora el conflicto suele venir de la convivencia entre trabajo propio y trabajo ajeno;
- la sincronización previa importa mucho más;
- retrasar demasiado la actualización de tu rama suele aumentar la fricción.

Contexto	Origen típico del conflicto
Local	Dos ramas tuyas modifican lo mismo
Remoto	Tu rama y cambios ya publicados en remoto compiten sobre la misma zona

> ⚠ **Precaución profesional**
>
> Cuanto más tiempo mantienes una rama sin sincronizarse con el proyecto compartido, más probable es que la integración sea costosa.

Referencias del subtema

- Git Book: conflictos de merge.
- Documentación oficial de Git.
- GitHub Docs.

---

15. `git revert`: deshacer sin reescribir historial compartido

En un contexto colaborativo, no siempre conviene “mover” o reescribir el historial. A veces necesitas deshacer un cambio manteniendo trazabilidad.

15.1. Qué hace `git revert`

`git revert` crea un nuevo commit que revierte el efecto de otro commit anterior.

15.2. Por qué es importante en trabajo compartido

En historiales que ya han sido publicados o compartidos, esta estrategia suele ser más segura conceptualmente que reescribir el pasado sin más.

15.3. Qué debes entender en esta fase

No necesitas dominar todavía todos sus matices, pero sí entender esta idea base:

- **reset** y **revert** no cumplen la misma función;
- `revert` es especialmente importante cuando el historial ya forma parte de un espacio compartido.

> ⓘ **Idea clave**
>
> `git revert` no borra el pasado: añade un nuevo paso que compensa uno anterior.

Referencias del subtema

- Documentación oficial: `git revert`.
- Git Book: deshacer cambios.
- Git Cheat Sheet oficial.

---

16. `git rebase`: concepto y cautela

`git rebase` es una herramienta potente, pero debe entenderse con cuidado.

16.1. Qué idea básica debes retener

`rebase` reubica commits sobre otra base del historial, generando una secuencia nueva de commits reescritos.

16.2. Qué aporta

Puede ayudarte a reorganizar o actualizar una línea de trabajo para que quede apoyada sobre un punto más reciente del proyecto.

16.3. Qué riesgo conceptual tiene

Como reescribe historial, no debes tratarlo como una operación inocua. En historiales ya compartidos, exige mucho más cuidado que en trabajo puramente local.

> ⚠ **Precaución**
>
> En esta fase te interesa comprender qué hace `rebase`, no usarlo como respuesta automática a cualquier problema de sincronización.

Referencias del subtema

- Documentación oficial: `git rebase`.
- Git Book: rebase.
- Explain Git with D3.

---

17. `git cherry-pick`: traer un commit concreto

A veces no necesitas integrar toda una rama, sino recuperar un commit concreto.

17.1. Qué hace

`git cherry-pick` aplica en tu rama actual el efecto de un commit específico seleccionado desde otra parte del historial.

17.2. Cuándo puede resultarte útil

Puede ser útil cuando:

- quieres recuperar una corrección concreta;
- no te interesa integrar toda la rama de origen;
- necesitas trasladar una pieza puntual de trabajo.

17.3. Qué debes entender por ahora

No es una herramienta de uso diario en todos los casos, pero sí conviene que la reconozcas porque aparece en flujos reales de mantenimiento y corrección puntual.

> **Idea práctica**
>
> `cherry-pick` te permite traer una pieza concreta del historial sin asumir toda una línea de trabajo completa.

Referencias del subtema

- Documentación oficial: `git cherry-pick`.
- Git Book: herramientas de Git.
- Git Cheat Sheet oficial.

---

18. `git stash` en contexto real de colaboración

Ya viste `stash` en local, pero en trabajo remoto su utilidad práctica se vuelve más evidente.

18.1. Qué problema resuelve aquí

Puede ayudarte cuando:

- necesitas cambiar rápidamente de contexto;
- quieres actualizar o revisar otra rama;
- no te conviene todavía hacer commit del trabajo en curso.

18.2. Qué debes evitar

No debes usar `stash` como sustituto de una rama bien planteada o de commits razonables. Sigue siendo una herramienta temporal.

18.3. Por qué aparece también en esta sesión

Porque al empezar a convivir con remotos, revisiones y cambios de contexto, las interrupciones del flujo se vuelven más frecuentes.

> **Criterio de uso**
>
> `stash` es útil para aparcar trabajo puntual. No debe convertirse en el contenedor habitual de un repositorio mal organizado.

Referencias del subtema

- Documentación oficial: `git stash`.
- Git Book: guardado rápido y limpieza.
- Git Cheat Sheet oficial.

---

19. Flujo de trabajo recomendado para esta sesión

A partir de todo lo anterior, el flujo general que vas a practicar puede resumirse así:

19.1. Secuencia operativa

1. clonas o abres un repositorio con remoto;
2. compruebas qué remoto existe;

3. revisas el estado de tu rama;
4. actualizas tu conocimiento del remoto;
5. partes de una base razonable;
6. creas una rama de tarea;
7. trabajas en local y haces commits con intención;
8. publicas la rama;
9. preparas revisión;
10. integras cuando corresponde;
11. resuelves conflictos si aparecen;
12. sincronizas tu entorno para continuar.

19.2. Hábitos que debes consolidar

Hábito	Por qué importa
No trabajar directamente sobre `main`	Reduce mezcla de cambios y facilita revisión
Entender antes de sincronizar	Evita usar `pull` a ciegas
Publicar ramas coherentes	Hace más legible el trabajo
Revisar antes de integrar	Mejora calidad y comunicación
Mantener ramas razonablemente actualizadas	Reduce fricción y conflictos costosos

✓ **Competencia que debes adquirir**

Al terminar esta sesión deberías ser capaz de explicar la relación entre local y remoto, diferenciar `fetch`, `pull` y `push`, trabajar con ramas publicadas y comprender la lógica de la colaboración básica con GitHub.

---

20. Análisis técnico avanzado

Esta sección amplía tu comprensión para que el trabajo remoto no quede reducido a una simple receta de comandos.

20.1. El remoto no reemplaza la estructura local

Tu trabajo sigue pasando por workspace, index y repositorio local. El remoto añade una nueva capa compartida, pero no elimina la arquitectura previa.

20.2. Sincronizar no significa descargar una carpeta nueva

Sincronizar implica relacionar historiales, referencias y ramas. Por eso `fetch`, `pull` y `push` tienen papeles distintos y no deben confundirse.

20.3. La rama deja de ser solo una herramienta privada

En local, una rama organiza tu trabajo. En remoto, además, se convierte en una unidad de publicación, revisión e integración.

20.4. La Pull Request formaliza una decisión técnica

La PR no es solo una comodidad de interfaz. Hace visible una propuesta de integración, mejora la revisión y ordena la conversación técnica.

20.5. `revert` y `rebase` no son intercambiables

Aunque ambos pueden aparecer en contextos de corrección o reordenación, responden a lógicas distintas. `revert` añade un nuevo paso que compensa el pasado; `rebase` reescribe una línea del historial.

20.6. Los conflictos remotos no son “otro tipo de conflicto”

Su naturaleza sigue siendo la misma: Git encuentra cambios incompatibles y no puede decidir solo cuál debe quedar. Lo que cambia es el contexto colaborativo donde aparecen.

20.7. Limitaciones de esta sesión

En esta sesión todavía no profundizas en:

- flujos avanzados con forks;
- automatizaciones de CI/CD;
- protección avanzada de ramas;
- gestión avanzada de releases;
- estrategias complejas de rebase colaborativo.

Esto es intencional. Primero necesitas consolidar el flujo colaborativo esencial.

---

21. Síntesis final

En esta sesión has ampliado Git desde el entorno local al entorno compartido. Has visto que un remoto no es una copia misteriosa “en internet”, sino otra instancia del repositorio con la que tu trabajo local necesita relacionarse de forma consciente.

También has trabajado la diferencia entre `clone`, `remote`, `fetch`, `pull` y `push`, y has visto que las ramas siguen siendo el eje de la organización cuando el proyecto se comparte. Publicar una rama no equivale a integrarla; la revisión y la Pull Request cumplen precisamente la función de ordenar esa transición.

Por último, has introducido herramientas y conceptos que cobrarán más importancia en situaciones reales: `revert`, `rebase`, `cherry-pick`, `stash` en contexto de colaboración y los conflictos remotos como consecuencia normal del trabajo concurrente.

La idea central que debes conservar es esta: **Git remoto no cambia la lógica básica de Git; la hace más exigente, porque ahora tus decisiones afectan también a otras personas y al estado compartido del proyecto**.

En la siguiente sesión, esta lógica se pondrá a prueba en **laboratorios individuales, por parejas y en grupo**, donde aplicarás el flujo completo en situaciones más realistas.

Referencias generales

Documentación oficial y manuales

- Git Book en español: `https://git-scm.com/book/es/v2`
- Documentación oficial de Git: `https://git-scm.com/docs/git/es`
- Documentación de GitHub: `https://docs.github.com/es`

Recursos visuales y de consulta

- Learn Git Branching: `https://learngitbranching.js.org/?locale=es_ES`
- Explain Git with D3: `https://onlywei.github.io/explain-git-with-d3/`
- Git Cheat Sheet interactiva: `https://ndpsoftware.com/git-cheatsheet.html`

Recursos complementarios

- Conventional Commits: `https://www.conventionalcommits.org/`
- Plantillas `.gitignore`: `https://github.com/github/gitignore`
- Git Exercises: `https://gitexercises.fracz.com/`